Introduction aux Bases de Données Relationnelles et Requêtes SQL de Base

Thème principal : Modèle Relationnel, Clés Primaires/Étrangères et Syntaxe SQL Explicite (SELECT, WHERE, INNER JOIN)

Synthèse exécutive : Ce cours pose les fondations de la ressource R1.05 en expliquant la transition essentielle des tableurs plats vers les systèmes de gestion de bases de données relationnelles (SGBD). Il détaille l'importance de la structuration des données sans redondance via les notions de clés primaires et de clés étrangères garantissant l'intégrité référentielle. Enfin, il introduit la syntaxe explicite normalisée du langage SQL (ANSI) pour la sélection, le filtrage et la jointure interne de tables.

1. Le Pourquoi : Des Tableurs aux Bases de Données Relationnelles

Dans les applications informatiques réelles, stocker des informations dans un fichier Excel ou un tableau plat pose rapidement des problèmes majeurs d'évolutivité et de fiabilité. Si une information (comme l'adresse d'un client) est répétée sur 50 lignes de commandes, la modifier nécessite 50 mises à jour distinctes. Si une seule ligne est oubliée, la base devient incohérente. Le modèle relationnel résout ce problème fondamental en séparant les entités distinctes (ex: 'Clients' d'un côté, 'Commandes' de l'autre) dans des tables dédiées et en établissant des liaisons logiques.

- Inconvénients des tableurs : Redondance de données, anomalies de mise à jour, risques majeurs d'incohérence.
- SGBD (Système de Gestion de Bases de Données) : Logiciel garantissant l'intégrité, la sécurité, la pérennité et l'accès concurrentiel aux données.
- Modèle Relationnel : Organisation des données sous forme de relations (tables) reliées logiquement entre elles pour éviter les répétitions.

Cas concret enseignant : *L'enseignant a illustré cela avec le système d'information bancaire : si un compte en banque stockait le nom du titulaire et son adresse sur chaque ligne de transaction, un simple déménagement exigerait de modifier des milliers d'historiques passés, créant un risque critique d'erreur de saisie.*

2. Clés Primaires et Clés Étrangères : L'Ancre de la Cohérence

Chaque n-uplet (ligne) d'une table doit pouvoir être identifié de manière unique. La Clé Primaire remplit ce rôle : elle ne peut jamais contenir de valeur nulle (NOT NULL) ni être dupliquée. Pour lier deux entités (par exemple associer un étudiant à son groupe de TD), on insère la clé primaire du groupe en tant que Clé Étrangère dans la table Étudiant. Le SGBD contrôle automatiquement toute opération d'insertion ou de modification pour garantir qu'un étudiant ne soit jamais affecté à un groupe inexistant.

- Clé Primaire (Primary Key - PK) : Attribut (ou ensemble d'attributs) identifiant de façon unique et non ambiguë chaque enregistrement d'une table.
- Clé Étrangère (Foreign Key - FK) : Attribut dans une table qui fait référence directe à la clé primaire d'une autre table.
- Contrainte d'Intégrité Référentielle : Règle stricte empêchant une clé étrangère de pointer vers un identifiant inexistant dans la table mère.

Cas concret enseignant : *L'enseignant a cité l'analogie du numéro de Sécurité Sociale (NIR) : même si deux personnes portent exactement le même nom et le même prénom au Pays Basque, leur identifiant unique empêche toute confusion ou doublon dans les systèmes administratifs.*

3. Extraction de Données avec SQL : SELECT, WHERE et INNER JOIN

Le langage SQL (Structured Query Language) est un langage déclaratif : on décrit au SGBD quel résultat on souhaite obtenir, et non les étapes algorithmiques pour y parvenir. Une requête de recherche repose sur la structure SELECT colonnes FROM table WHERE condition. Pour combiner deux tables reliées, la bonne pratique ISO/ANSI exige l'utilisation de la clause INNER JOIN table2 ON table1.fk = table2.pk.

- Projection (SELECT) : Spécification explicite des colonnes à extraire et à afficher.
- Filtrage (WHERE) : Application de critères logiques pour cibler uniquement les lignes souhaitées.
- Jointure Explicite (INNER JOIN ... ON) : Combinaison de deux tables basées sur la correspondance entre la clé étrangère et la clé primaire.

Cas concret enseignant : *L'enseignant a vivement mis en garde la promotion : 'N'utilisez jamais la syntaxe implicite avec une virgule dans le FROM (ex: FROM Etudiant, Groupe WHERE Etudiant.id_g = Groupe.id). Utilisez TOUJOURS la syntaxe explicite JOIN ... ON. Le produit cartésien implicite est pénalisé aux examens et banni en entreprise.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne doit-on pas utiliser SELECT * dans un code applicatif ou un examen ?
Réponse du professeur : L'utilisation de SELECT * est une mauvaise pratique majeure. Elle force le SGBD à lire et transférer l'intégralité des colonnes, ce qui dégrade inutilement les performances réseau et mémoire. De plus, cela rend le code applicatif fragile : si la structure de la table change, l'application peut planter. Il faut toujours expliciter la projection des colonnes nécessaires.

Question étudiant : Que se passe-t-il si j'essaie de supprimer un groupe qui contient encore des étudiants ?
Réponse du professeur : Par défaut, le SGBD bloque la suppression et renvoie une erreur de violation d'intégrité référentielle. Il interdit la création de n-uplets orphelins (des étudiants pointant vers un groupe qui n'existe plus). Pour autoriser la suppression, il faudrait expliciter une action en cascade (ON DELETE CASCADE), mais sans cela, le SGBD protège la cohérence des données.

■ Exercice & Démonstration d'Application

Énoncé : On considère deux tables : Etudiant(id_etudiant, nom, prenom, #id_groupe) et Groupe(id_groupe, nom_groupe, promo). Écrire la requête SQL conforme aux normes ISO/ANSI permettant d'extraire le nom et le prénom des étudiants appartenant à la promo 'BUT1-INFO' du groupe nommé 'A1'.

Résolution pas à pas :
Étape 1 : Identifier les tables cibles -> Etudiant (alias e) et Groupe (alias g). Étape 2 : Définir la jointure explicite sur la clé étrangère -> INNER JOIN Groupe g ON e.id_groupe = g.id_groupe. Étape 3 : Poser les critères de filtrage dans la clause WHERE -> g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1'. Étape 4 : Définir la projection explicite des colonnes demandées -> SELECT e.nom, e.prenom.

Résultat : SELECT e.nom, e.prenom FROM Etudiant e INNER JOIN Groupe g ON e.id_groupe = g.id_groupe WHERE g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1'; Commentaire : Cette requête est optimale, lisible, respecte la norme ANSI et évite tout produit cartésien implicite.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la propriété fondamentale d'une clé primaire (Primary Key) ?

R : Une clé primaire identifie de façon unique chaque n-uplet d'une table. Elle ne peut en aucun cas contenir de valeur NULL ni posséder de doublons.

Q : À quoi sert la clause INNER JOIN en SQL et comment s'articule-t-elle ?

R : Elle effectue une jointure interne entre deux tables en associant les lignes qui satisfont la condition d'égalité spécifiée dans la clause ON (généralement entre une clé étrangère et une clé primaire).

Q : Pourquoi la syntaxe 'FROM TableA, TableB WHERE TableA.id = TableB.id_a' fait-elle perdre des points en examen ?

R : Parce qu'il s'agit d'une jointure implicite issue du produit cartésien. Elle mélange le filtrage des lignes et la liaison des tables, ce qui est contraire à la norme ANSI modernisée et favorise les erreurs d'oubli de condition de jointure.